

0.80.2. x et y rangés ensemble : struct — une fonction dans un fichier

Le 0.80.1, en mieux : l'intérieur de la boucle devient une fonction, `tour_de_jeu()`, rangée dans `jeu.h`. La boucle n'a plus qu'une ligne.

1. Ce qu'il faut comprendre

C'est le 0.80.1, en mieux : au lieu de verser des lignes en vrac au milieu de la boucle, on les range dans une fonction, `tour_de_jeu()`, dans le fichier `jeu.h`.

Ce qui est nouveau ici : `void`. Au 0.33, notre fonction **rendait** une valeur (`uint8_t`, puis `return colonne;`). Celle-ci ne rend rien : elle **fait** quelque chose. On l'écrit `void tour_de_jeu() { ... }` : `void` veut dire « rien », et les parenthèses vides veulent dire qu'elle ne reçoit rien.

Le `#include "jeu.h"` revient tout en haut, avant `main`, comme au 0.67 : il apporte la **définition** de la fonction. La boucle, elle, se contente de l'**appeler** : `tour_de_jeu();`, une ligne.

Pourquoi c'est mieux : en ouvrant `jeu.h`, on voit une fonction complète, avec son nom et ses accolades : on sait ce qu'elle est. Et `main` se lit comme une phrase : « tant que le jeu tourne, fais un tour de jeu ».

L'ordre compte : `joueur` est déclaré **avant** le `#include "jeu.h"`, parce que la fonction s'en sert. Une variable doit exister avant qu'on écrive quelque chose qui l'utilise.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <deplace_croix>` → [0.66.3. La fonction `deplace\_croix\(\)` — suivre la croix, case par case](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <nombre>` → [0.8.3. La fonction `nombre\(\)` — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1245 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      65 o  ← la console, à l'allumage — la mise en route de la console
deplace_croix() 217 o  ← tour_de_jeu() — une fonction fournie par le langage
tour_de_jeu()   55 o  ← main() — ta fonction
main()          16 o  ← démarrage — ta fonction
VBlank          8 o  ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage   28 o  ← image() — attend l'image suivante
AttendreVBlank  16 o  ← démarrage — attend que l'écran soit libre
AttendreAcces   12 o  ← EffacerCases, EcrireNombre — attend de pouvoir écrire une case
AdresseCase     10 o  ← effacer(), poser() — trouve l'adresse d'une case de l'écran
EffacerCases    12 o  ← effacer() — efface des cases
EcrireNombre    33 o  ← nombre() — écrit un nombre en chiffres
CopierTuiles    18 o  ← démarrage — copie les lettres et les dessins en mémoire vidéo
```

```

EffacerCarte      14 o ← démarrage — vide les deux cartes de l'écran
LireManette       37 o ← bouton() — lit les huit boutons
Tuiles           704 o ← CopierTuiles — la police entière
Lettres gravées : toutes (poser()) reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, Diviser, Reste, Multiplier, DecalerGauche,
DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes
Fonctions du langage appelées :
nombre()         × 2    22 o sur place, dans tour_de_jeu() + EcrireNombre (33 o)
bouton()         × 4    20 o sur place, dans deplace_croix() + LireManette (37 o)
effacer()        × 1    15 o sur place, dans deplace_croix() + AdresseCase (10 o), EffacerCases (12 o)
poser()          × 1    13 o sur place, dans deplace_croix() + AdresseCase (10 o)
image()          × 1     3 o sur place, dans tour_de_jeu() + AttendreImage (28 o)
images()         × 1     2 o sur place, dans deplace_croix()
Tes fonctions appelées :
deplace_croix() × 1    21 o sur place, dans tour_de_jeu()
tour_de_jeu()  × 1     3 o sur place, dans main()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// Le nouveau type : la colonne et la ligne sous un seul nom (le 0.80).

#include <deplace_croix> // une tuile qui suit la croix, case par case
#include <ALPHABET>      // les lettres de la police : ALPHABET[0] est le A
#include <nombre>        // écrit un nombre en chiffres

```

2. UN ENREGISTREMENT — POSITION

```

struct Position { uint8_t x, y; };

```

3. UNE VARIABLE — JOUEUR

```

Position joueur; // joueur.x et joueur.y

// La fonction tour_de_jeu : voir l'onglet jeu.h. Elle se sert de joueur,
// déclaré juste au-dessus : l'ordre compte.
#include "jeu.h"

```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```

int main() {
    joueur.x = 10; // départ : (10, 0)
    joueur.y = 0;

    while (true) {
        tour_de_jeu(); // le changement : UNE ligne, l'appel de la fonction
    }
}

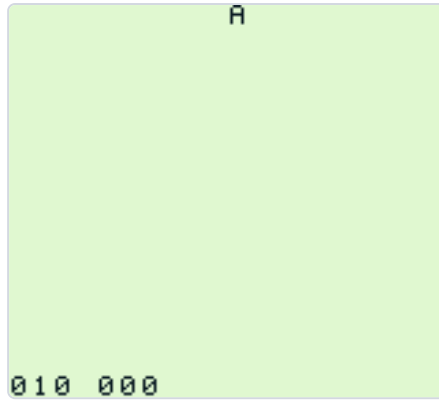
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Exactement le 0.80 : un A qui suit la croix. La boucle ne contient plus qu'un appel : `tour_de_jeu()`.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le A part de (10, 0)
- ✓ la croix le déplace, depuis l'autre fichier (6, 2)
- ✓ il est à la place que disent `joueur.x` et `joueur.y`

Ensuite : **0.81. Ramasser une pièce : le P** — chapitre 0 : Avant tout. · `gameboy3` — écrire une cartouche Game Boy en C++